



Organigramme des Algorithmes Detecteur Pro

Architecture modulaire d'un système de détection d'intrusion client-side — découplage par événements JavaScript, persistance locale et interface temps réel.

Vue d'ensemble

Une architecture en 5 modules indépendants



Principal

Orchestration globale et cycle de vie du système



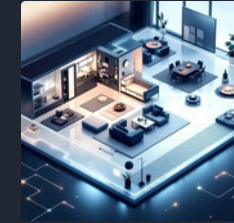
Détection

Analysedes capteurs virtuels et déclenchement d'alertes



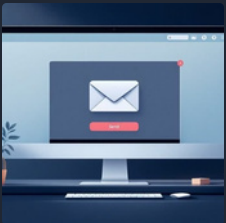
Historique

PersistanceJSON via LocalStorage



Simulation

Rendu visuel temps réel de la maison surveillée



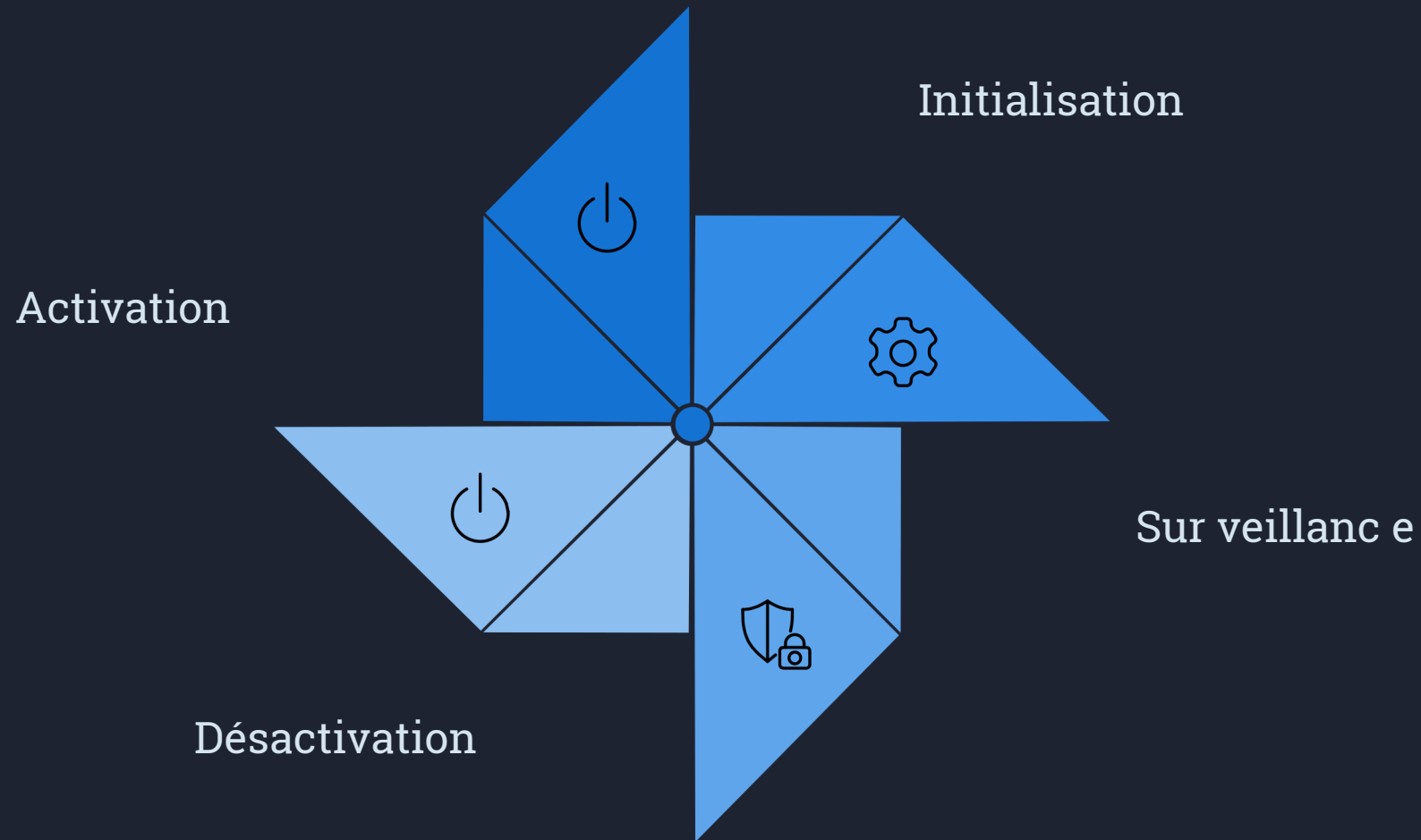
Contact

Formulaire validé JS avec envoi via EmailJS

☐ Les modules communiquent exclusivement via `CustomEvent` JavaScript — aucune dépendance directe entre eux.

Algorithme Principal

Cycle de vie du système



La boucle principale tourne en continu tant que le système est actif, simulant le comportement d'un vrai système de sécurité embarqué. Toute intrusion détectée déclenche alertes et enregistrement avant de reprendre la surveillance.

De la lecture capteur à l'alerte

01

Lecture des zones

État de chaque capteur virtuel
interrogé à chaque cycle

Capteur inactif

Passage direct à la boucle
suivante — aucune action
déclenchée

02

Analyse d'intrusion

Si capteur actif → confirmation de
l'intrusion par analyse

Intrusion non confirmée

Retour en surveillance sans
écriture dans l'historique

03

Déclenchement alertes

Alertes visuelles et sonores émises
immédiatement

Intrusion confirmée

Chaîne complète alertes →
historique → mise à jour
simulation

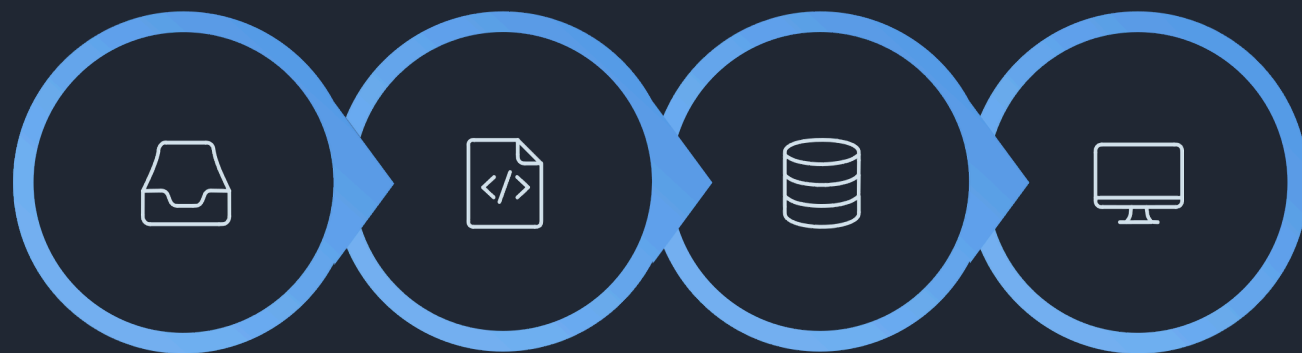
04

Notification Historique

Événement transmis au module
Historique via CustomEvent



Persistance locale des événements



Réception

Création
JSON

L o c a lStorage

Affichage

Pourquoi LocalStorage ?

La persistance côté navigateur garantit que l'historique des intrusions survit à un rechargement de page, sans serveur ni base de données externe.

Sans serveur
100% client-side

JSON natif
Sérialisation simple

Temps réel
UI synchronisée



Module Simulation

Visualisation temps réel de la maison

1

Chargement

Représentation graphique de la maison et positionnement des détecteurs virtuels

2

Écoute événements

Abonnement aux `CustomEvent` émis par le module Détection

3

Animations

Mise à jour immédiate des visuels — zones actives, alertes colorées, état des capteurs

Formulaire validé et envoi EmailJS



Saisie

Validation

Envoi API

Réponse

Validation JavaScript

Tous les champs sont contrôlés côté client avant tout appel réseau. En cas d'erreur, le formulaire reste visible avec les messages contextuels.

Validation synchrone

Aucun appel API si les champs sont invalides

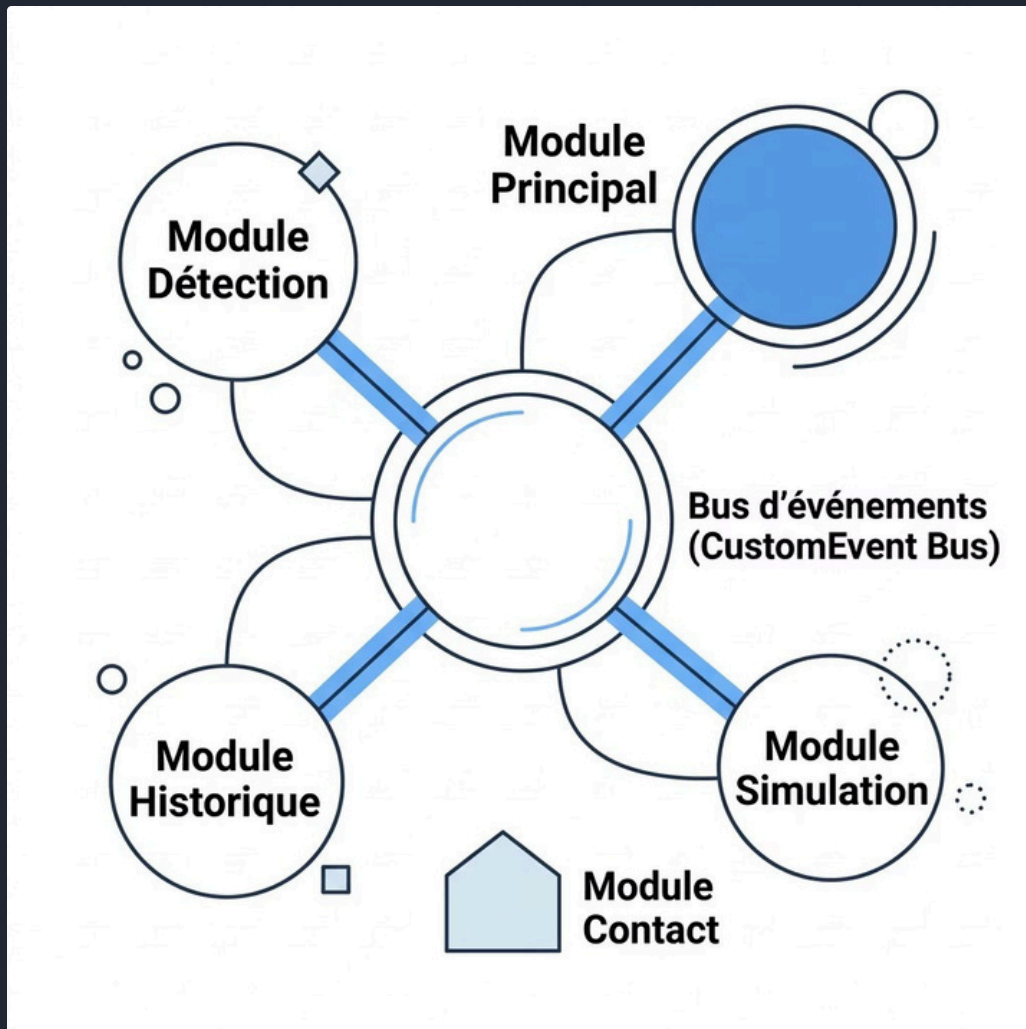
EmailJS

Envoi d'email sans back-end, via SDK JavaScript

Feedback utilisateur

Confirmation ou erreur affichée après réponse de l'API

Découplage par CustomEvent



Pourquoi les événements ?

Chaque module émet ou écoute des CustomEvent sans connaître l'identité de ses consommateurs. Ce découplage fort garantit une maintenabilité et une testabilité maximales.

Testabilité unitaire

Chaque module peut être instancié et testé isolément

Extensibilité

Ajouter un module = s'abonner aux événements existants

Zéro couplage fort

Aucune référence directe entre modules

Bilan technique

Points clés pour les équipes front-end



Modularité

5 modules indépendants, chacun avec une responsabilité unique et clairement délimitée



Persistence

LocalStorage pour l'historique — pas de serveur, pas de configuration infrastructure



Événementiel

`CustomEvent` comme seul canal de communication inter-modules



Boucles continues

Détection & Simulation actives en permanence, simulant un vrai système embarqué



Detecteur Pro – Ce qu'il faut retenir

Architecture

5 modules découplés communiquant via CustomEvent, sans aucune dépendance directe. Chaque module est testable, remplaçable et extensible indépendamment.

Technologies clés

- JavaScript natif (CustomEvent API)
- LocalStorage pour la persistance
- EmailJS pour l'envoi sans back-end

Prochaines pistes d'amélioration

→ Notifications push

Intégration de l'API Web Push pour alertes hors-navigateur

→ Tests unitaires

Couverture Jest sur chaque module grâce à l'architecture découplée

→ Synchronisation cloud

Remplacement du LocalStorage par une API REST ou Firebase